CS3021 Introduction to Data Structures and Intermediate Programming

Sets and Maps: Collection ADTs

C++ Containers (1)

- A C++ container is characterized by having useful types and operations for a collection of elements.
- Provide a sequence of elements (`begin()`:`end()`).
- Provide copy operations that copy all elements.
- Provide move operations that move all elements.
- Provide iterator types called `iterator`, `const_iterator`.
- Iterators provide `*`, `++` (prefix and postfix), `==`, and `=` (assignment) with the appropriate semantics.
- List iterators also provide `--` (prefix and postfix) for moving backward in a sequence.

C++ Containers (2)

- Provide comparison operators (`==`, `!=`, `<`, `<=`, `>`, `>=`) that compare two containers' elements.
- Containers use lexicographical ordering for comparisons; that is, they compare their elements in order starting with the first.
- Vector iterators also provide `--`, `[]`, `+`, and `-`, called random-access iterators.
- Provide `insert()` and `erase()`, `front()`, and `back()`, `size()`, `swap()`, etc.
- Vector and map containers also provide subscripting operator `[]`.

Some C++ Container Classes

- vector - a contiguously allocated sequence of elements; use it as the default container.
- list - a doubly-linked list; use to insert and delete elements without moving existing elements.
- set - a balanced ordered tree; use it when you need to keep track of individual values.
- map - a balanced ordered tree; use it when you need to access elements by value.
- unordered_set - like unordered_map, but just with values, not (key,value) pairs.
- unordered_map - a hash table; an optimized version of map; use for large maps when you need high performance and can devise a good hash function.
- And more...

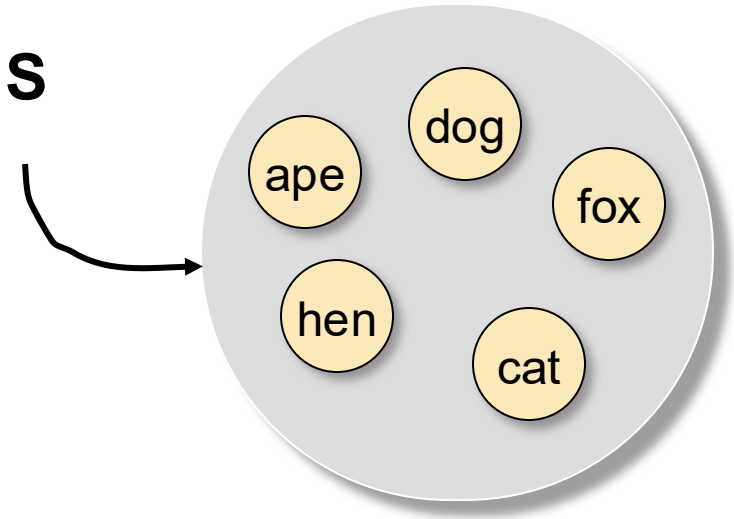
Other "Like" Containers

- These other data types provide much of what is required in a standard container, but not everything.
  - Not all are in the C++ standard.
- T[n] - built-in array; no `size()` or other member functions; a container, such as vector, string, or array, is preferred over a built-in array, given a choice.
- array - fixed-size array that doesn't have most of the problems related to the built-in arrays.
- string - holds only characters but provides operations useful for text manipulation, such as concatenation (`+` and `+=`); prefer the standard string to other strings.
- istream - read from like a container, but not write.
- ostream - write to like a container, but not read.

The Set ADT

- A *set* ADT is an *unordered* collection of objects with *no duplicates*.

S = { ape, hen, dog, cat, fox }



Set Operations

- Common operations on *set* ADT include:
  - Testing for membership (using unique *key*)
  - Adding elements
  - Removing elements
  - Union
  - Intersection
  - Difference

Set Implementations in C++ (1)

- C++ provides two associative container class templates for *std::set* and *std::unordered_set* that each contain a set of unique objects of type *key*.
- Internally, *set* elements are sorted using a comparison function of their *key*, and are stored using Red-Black trees (self-balanced binary trees… we'll cover later).
- Conversely, *unordered_set* elements are not sorted in any order, but are organized into buckets according to a hash of their *key*, stored in a hash table.
  - This allows faster access to individual elements, since a hash refers to the exact bucket the element is placed into.
- Set elements may not be modified since *key* is immutable and changing it would corrupt the container.

Set Implementations in C++ (2)

- A *set* stores elements in a sorted order, and operations such as insertion, deletion, and accessing operation take longer than with an *unordered_set*. Use when:
  - You need ordered data, such as for printing elements.
  - You need predecessor/successor information.
  - You want to use functions such as binary_seach, lower_bound, etc.
- An *unordered_set* stores elements in an arbitrary order, and insertion, deletion, and access operations are faster due to the use of hashing. Use when:
  - You need a collection of distinct elements, but no ordering is required.
  - You need faster element access, but no set traversal.

Declaring and Initializing a Set

- We can declare and initialize a set using the set class template constructor as follows (must use `#include <set>`):

```
int main()
{
  set<int> set1;   // using default constructor (empty set)
  set<int> set2 = {5, 3, 2, 4, 1};  // using initializer list

  for (auto i : set2) { cout << i << " "; } cout << endl;
  return 0;
}  //end main
```

sets1.cpp

Inserting Elements to a Set

- New set elements (*keys*) are added using the `insert()` method.
  - If the element already exists, it will not be added.

```
int main()
{
  set<int> set1 = {2, 4, 1};
  set1.insert(3);
  set1.insert(2);  // 2 will not be added
  for (auto i : set1) { cout << i << " "; } cout << endl;
  return 0;
}  //end main
```

sets2.cpp

Updating or Deleting Elements in a Set

- You cannot update the set elements (*keys*) once they are stored in the set.
- However, set elements (*keys*) may be deleted using the `erase()` method.
  - If the element doesn't exist, it will be ignored.

```
int main()
{
  set<int> set1 = {2, 4, 1};
  set1.delete(2);
  set1.delete(3);  // this call will be ignored
  for (auto i : set1) { cout << i << " "; } cout << endl;
  return 0;
}  //end main
```

sets3.cpp

Iterators (1)

- Sets (and other container) allow array-like indexing (e.g. `myset[i]`, `myset.at(i)`) that finds the correct data, while giving the illusion that data is contiguous in memory (like an array).
- To iterate over the whole container, you could use a counter variable and array indexing, but it can be more efficient to keep an internal pointer to the next item and update it iteratively.
- C++ STL containers define *helper* classes called *iterators* that store such internal pointers to help iterate over each container element or find an item in the container.

Iterators (2)

- Iterators are a class type defined in the scope of each container type.
- Initialize them with `objname.begin()`, check whether they are finished by comparing with `objname.end()`, and move to the next item with ++ operator.

```
#include <iostream>
#include <set>
using namespace std;
int main()
{
  set<int> my_set(5); // 5 = initial size
  for(int i=0; i < 5; i++){
    my_set.insert(i);
  }
  set<int>::iterator it;
  for(it = my_set.begin() ; it != my_set.end(); ++it){
    do something...
  }
}
```

```
// set.h
template<class T>
class set
{
  class iterator {

  };
};
```

Iterators (3)

- An iterator variable has the same semantics as a pointer to an item in the container.
  - Use `*` to dereference and get the element (*key*).
  - Since we're storing integers in the set below, the iterator acts and looks like an `int*`.

```
#include <iostream>
#include <set>
using namespace std;
int main()
{
  set<int> my_set(5); // 5 = initial size
  for(int i=0; i < 5; i++){
    my_set.insert(i);
  }
  for(set<int>::iterator it = my_set.begin() ; it != my_set.end(); ++it){
     cout << *it << endl;
  }
  return 0;
}
```

Iterator Tips

- Think of an iterator variable as a pointer… when you declare it, it initially points at nothing.
- The `begin()` method returns the address of the first item in the container and assigns that to the iterator.
- The `end()` method returns the address *after* the last item in the container (i.e., off the end of the collection), so as long as the iterator is less than or not equal to, you're still safely within the container.

Iterators and Ranges

- Iterators can be a source of errors:

```
sort(s1.end(), s1.start());      // Oops!
sort(s1.start(),s2.end());       // Oops!
```

- Ranges can be used instead.
  - They can more precisely say what's intended, e.g.:

```
ranges::sort(s1);
```

  - If this doesn't make sense the compiler will complain.
- Can define a range in three ways:
  - `{begin, end}` a pair of iterators
  - `{begin, length}` an iterator and # of elements
  - `{begin, pred.}` an iterator and predicate to determine if end has been reached

Membership in a Set

- Set provides fast membership search by *key* using the `find()` method.
- The method returns an iterator to the element if found; otherwise, returns the `end()` iterator.

```
int main()
{
  set<int> set1 = {2, 4, 1, 5, 3};
  auto item1 = set1.find(3);
  auto item2 = set1.find(6);  // not in set1
  if (item1 != set1.end()) cout << *item1 << endl;
  if (item2 != set1.end()) cout << *item2 << endl;
  return 0;
}  //end main
```

sets4.cpp

Accessing Elements in a Set

- You can increment/decrement an iterator returned by the `begin()` or `end()` methods to access an element of a set, then use `next()` or `advance()` to move through the set.

```
int main()
{
  set<int> set1 = {2, 4, 1, 5, 3};
  auto item1 = set1.begin();    // access first set element
  auto item2 = next(item1, 2);  // move to third element
  cout << *item1 << " "; << *item2 << endl;
  return 0;
}  //end main
```

sets5.cpp

Set Equality, Relational Operations

- We can compare the contents of two *set* objects for their equality, and relative to one another.
  - The operators ==, !=, <, <=, >, >= are all defined as expected for *std::set*.
  - The "spaceship operator" <=> tests equality and relational comparison (added in C++ 20).
- The equality/inequality operators check if all elements of the two sets are the same (or not).
- The relational operators compare the contents of the two sets lexicographically:
  - E.g., {2, 3, 1} < {1, 4, 2} is *true.*

sets6.cpp

Common Operations on Sets

- The C++ STL provides set functions union, intersection, and difference using *set* container.
- To find the *set union* of set1 and set2

```
set_union(set1.begin(), set1.end(), set2.begin(),
set2.end(), output_iterator);
```

- To find the *set intersection* of set1 and set2

```
set_intersection(set1.begin(), set1.end(), set2.begin(),
set2.end(), output_iterator);
```

- To find the *set difference* of set1 and set2

```
set_difference(set1.begin(), set1.end(), set2.begin(),
set2.end(), output_iterator);
```

sets7.cpp

The Map ADT (1)

- A *map* ADT is an <u>unordered</u> *collection* of *key-value* pairs.
  - Both the *key* and the *value* can be any type of object.
- A *map* cannot contain duplicate *keys*, i.e., the *keys* form a *set* ADT.
- A *map* may have duplicate *values*, however…
- Each *key* in a *map* can map to <u>*at most*</u> one value.

The Map ADT (2)

- Think of a *map* like an array, where the index is not limited to being only an integer, e.g., the *key* could be a string.
  - Although map *keys* form a *set*.
- A *map* is a good structure when you want to implement efficient lookup of a *value* indexed by some *key*:
    - dictionary (*key*: word, *value*: definition)
    - phone book (*key*: name, *value*: phone_num)
    - address book (*key*: SSN, *value*: Person)

Map Implementations in C++

- As with sets, C++ provides two associative container class templates for *std::map* and *std::unordered_map* that each contain a map of unique *key:value* pairs.
- *map* elements are sorted by their *keys*, and stored using Red-Black balanced binary trees.
- *unordered_map* elements are stored in a hash table by hashing their *key*, and are not sorted in any order, but are faster in lookup.
- Map *keys* may not be modified since they are immutable, like with Set *keys*.

Map Operations in C++

- Since a *map* is a specialized *set*, many of the *set* methods we saw earlier are provided (must use `#include <map>`):

```
map<int, string> map1;   // default constructor (empty map)
map<int, string> map2 = {{1, "first"}, {2, "second"},
                         {3, "third"}};  // initializer list
map1.insert({1, "First"});
map2.insert({2, "Second"});  // ignored, can't reinsert
map2.erase(2);
map2.erase(2);  // ignored, no key 2
for (auto i : map2)  // traverse map to print
  cout << i.first << " " << i.second << endl;
```

maps1.cpp

Map Operations in C++

- Since *map values* are mutable, additional operations are provided to support this:

```
map<int, string> map1 = {{1, "first"}, {2, "second"},
                         {3, "third"}};
auto it1 = map1.find(2);  // returns iterator to it1
if (it1 != map1.end())    // output using the iterator
  cout << it1->first << " " << it1->second;
else cout << "Key not found!";

auto it2 = map1.find(4);  // no Key 4 to return

map1.at(1) = "First";     // modify a map Value by its Key
```

maps2.cpp

Questions?